Project 2: Volatility of the South African Rand with ARCH and GARCH Models

Section 1: Conditional variance models

Eden

Table of contents

Executive summary 1

1 Introduction and research question 2

2 Data 2
- 2.1 Source and data statement . . . 2
- 2.2 Reading the data and computing returns . . . 3

3 Exploratory data analysis (Box and Jenkins step 1) 3
- 3.1 The exchange rate and its returns . . . 3
- 3.2 Distribution of returns . . . 5
- 3.3 Dependence in returns and in squared returns . . . 6

4 Testing for ARCH effects 7

5 Models 8
- 5.1 The Box and Jenkins loop for returns . . . 8
- 5.2 Candidate models . . . 9
- 5.3 Estimates . . . 9
- 5.4 Residual diagnostics (Box and Jenkins step 5) . . . 11

6 My own implementation of GARCH(1,1) 13
- 6.1 A small simulation check . . . 16

7 Out-of-sample evaluation: Value-at-Risk backtest 17
- 7.1 Design . . . 17
- 7.2 Backtest tests . . . 18
- 7.3 Variance forecast accuracy . . . 20

8 Volatility forecasts 22

9 Discussion 23
- 9.1 Key findings . . . 23
- 9.2 Issues and limitations . . . 24
- 9.3 What I would do differently . . . 24

Executive summary

Executive summary

Title: Volatility of the South African rand with ARCH and GARCH models
Section and topic: Section 1. Conditional variance models: stylised facts of returns, testing for ARCH effects, ARCH, GARCH and asymmetric GARCH models, volatility forecasting.
Type of project: Implementation (own maximum likelihood code for GARCH(1,1), checked against `rugarch` and by a small simulation study) and model comparison on real-world data. The Value-at-Risk backtest is an idea not covered in the lectures.
Question: Do daily USD/ZAR exchange rate returns show volatility clustering, and which conditional variance model gives the most reliable one-day-ahead risk forecasts?
Data: Real-world, publicly available data: the South African rand to US dollar spot exchange rate (FRED series DEXSFUS, Board of Governors of the Federal Reserve System), daily, January 2010 to September 2026. Full source details are in Section 2.1.
Methods: Exploratory analysis of the stylised facts; Ljung and Box and ARCH LM tests; the Box and Jenkins loop applied to the mean and the variance equation; ARCH(1), GARCH(1,1) with normal and Student t errors, and GJR-GARCH(1,1) with Student t errors; my own implementation of the GARCH(1,1) likelihood; a rolling out-of-sample evaluation of 1000 one-day-ahead forecasts, scored by Value-at-Risk backtests (Kupiec and Christoffersen tests), quantile loss and QLIKE, against historical simulation and constant variance benchmarks.
Key findings: The returns show all the stylised facts: almost no autocorrelation in the returns themselves, strong autocorrelation in squared returns, and fat tails. ARCH(1) fails the residual diagnostics, while the GARCH(1,1) models capture the clustering with a very persistent variance. Student t errors fit far better than normal errors. The detailed results of the backtest, including which models give correctly calibrated VaR, are in Section 9.1.
What I would do differently: Use a longer test period so that the 1% VaR tests have more power, and compare GARCH with a regime-switching model of volatility (Project 4).
Detailed analysis: the sections below.

1 Introduction and research question

Exchange rate returns behave very differently from the seasonal series in Project 1. The level of the rand is close to a random walk, so its changes are almost unpredictable, but the *size* of those changes is not: calm periods and turbulent periods come in clusters. ARIMA models describe the conditional mean and assume a constant variance, so they cannot capture this. ARCH and GARCH models add a second equation for the conditional variance.

The rand is a useful series for this because it is one of the most traded and most volatile emerging market currencies, and its history since 2010 includes several clear shocks, for example the dismissal of the finance minister in December 2015 and the start of the COVID-19 pandemic in March 2020.

Research question. *Do daily USD/ZAR returns show volatility clustering, and which conditional variance model (ARCH, GARCH or asymmetric GARCH, with normal or Student t errors) gives the most reliable one-day-ahead risk forecasts?*

"Reliable" is made precise through Value-at-Risk (VaR). The one-day VaR at level p is the p quantile of the forecast distribution of tomorrow's return. A good model should be exceeded on close to a fraction p of days, and those exceedances should not cluster.

2 Data

2.1 Source and data statement

Data source at a glance

Publisher	Board of Governors of the Federal Reserve System (US), via FRED, Federal Reserve Bank of St. Louis
Series	DEXSFUS: South African Rand to US Dollar Spot Exchange Rate (rand per one US dollar), noon buying rates in New York
Frequency	Daily (business days), not seasonally adjusted
Where to get it	https://fred.stlouisfed.org/series/DEXSFUS
File used	`usdzar_fred.csv`, downloaded by the code below from the FRED CSV link
Period analysed	1 January 2010 to 30 September 2026
Date downloaded	*[add the date you first rendered this file]*
Licence and privacy	Public domain official statistics; no personal information, so POPIA does not apply

Reference: Board of Governors of the Federal Reserve System (US) (2026) *South African Rand to U.S. Dollar Spot Exchange Rate [DEXSFUS]*. Retrieved from FRED, Federal Reserve Bank of St. Louis. Available at: https://fred.stlouisfed.org/series/DEXSFUS (Accessed: *[date]*).

2.2 Reading the data and computing returns

The code downloads the series the first time the document is rendered and saves it as `usdzar_fred.csv`. If the download fails (for example without an internet connection), download the CSV manually from the FRED page above and save it in the same folder as this file under that name.

```
if (!file.exists(data_file)) {
  fred_url <- paste0("https://fred.stlouisfed.org/graph/fredgraph.csv?id=DEXSFUS",
                     "&cosd=", start_date, "&coed=", end_date)
  download.file(fred_url, data_file, mode = "wb")
}

fx_raw <- read.csv(data_file, na.strings = c(".", ""))
names(fx_raw) <- c("date", "rate")

fx <- fx_raw |>
  mutate(date = as.Date(date), rate = as.numeric(rate)) |>
```

```
  filter(!is.na(rate), date >= start_date, date <= end_date) |>
  arrange(date) |>
  mutate(r = 100 * (log(rate) - lag(log(rate)))) |>
  filter(!is.na(r)) |>
  mutate(day = row_number())

# tsibble indexed by trading day (the calendar dates are irregular
# because of weekends and US public holidays)
fx_ts <- fx |> as_tsibble(index = day)

r <- fx$r
n <- length(r)
```

Days on which FRED records no rate (US public holidays) are dropped, so returns are measured from one trading day to the next. The return is defined as

$$r_t = 100 \times (\log P_t - \log P_{t-1}),$$

where P_t is the number of rand per US dollar. A **positive return means the rand weakened** against the dollar. The analysis uses 4,188 daily returns from 05 January 2010 to 30 September 2026.

3 Exploratory data analysis (Box and Jenkins step 1)

3.1 The exchange rate and its returns

```
events <- as.Date(c("2015-12-09", "2020-03-11"))

p_level <- ggplot(fx, aes(date, rate)) +
  geom_line(colour = "steelblue4", linewidth = 0.3) +
  geom_vline(xintercept = events, linetype = "dashed", colour = "grey40") +
  labs(x = NULL, y = "ZAR per USD")

p_ret <- ggplot(fx, aes(date, r)) +
  geom_line(colour = "grey30", linewidth = 0.2) +
  geom_vline(xintercept = events, linetype = "dashed", colour = "grey40") +
  labs(x = NULL, y = "Daily return (%)")

p_level / p_ret
```

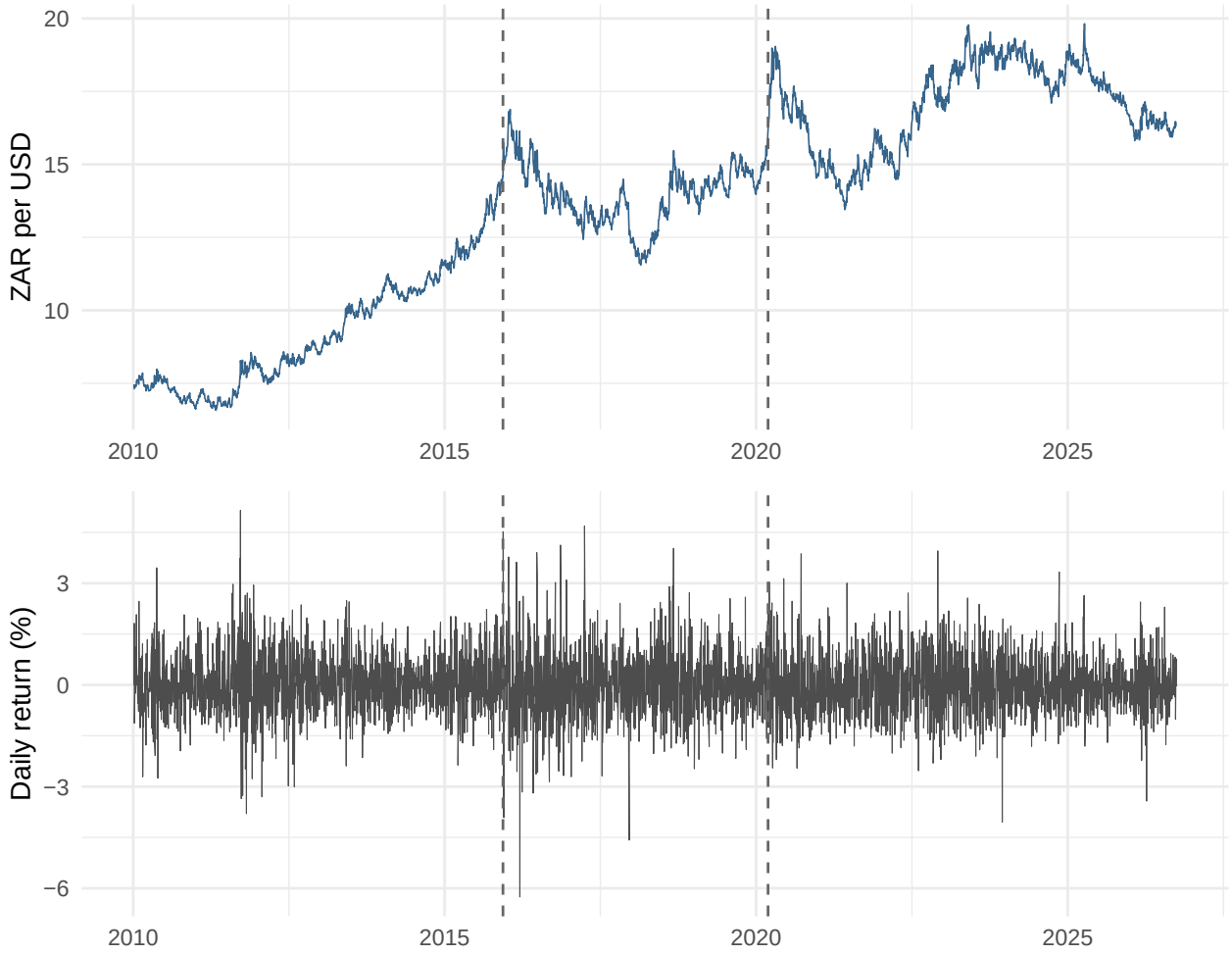


Figure 1: Top: rand per US dollar. Bottom: daily log returns (%). The dashed lines mark 9 December 2015 (dismissal of the finance minister) and 11 March 2020 (WHO declares COVID-19 a pandemic).

```
skew   <- mean((r - mean(r))^3) / sd(r)^3
kurt   <- mean((r - mean(r))^4) / sd(r)^4
jb     <- n / 6 * (skew^2 + (kurt - 3)^2 / 4)
jb_p   <- pchisq(jb, df = 2, lower.tail = FALSE)
ann_vol <- sd(r) * sqrt(252)

biggest <- fx |> slice_max(abs(r), n = 1, with_ties = FALSE)

tibble(
  `Trading days` = n,
  Mean = mean(r), `Std. dev.` = sd(r),
  Minimum = min(r), Maximum = max(r),
  Skewness = skew, `Excess kurtosis` = kurt - 3,
  `Annualised volatility (%)` = ann_vol
) |>
  kable(digits = 3)
```

Table 2: Summary statistics of daily returns (%). Excess kurtosis is kurtosis minus 3, so it is 0 for a normal distribution.

Trading days	Mean	Std. dev.	Mini-mum	Maxi-mum	Skew-ness	Excess kurtosis	Annualised volatility (%)
4188	0.019	0.939	-6.257	5.146	0.219	1.882	14.905

The rand weakened from about 7.31 to 16.42 per dollar over the period, which shows up as a small positive mean return of 0.0194% a day. This is tiny next to the daily standard deviation of 0.939%, which is the first stylised fact: over one day the mean is close to zero and nearly all the action is in the variance. The annualised volatility is 14.9%. The return series in Figure 1 shows the second stylised fact very clearly: **volatility clustering**. Large moves come in bursts, most obviously around the two marked events, separated by long calm stretches. The largest single move in the sample is -6.26% on 17 March 2016.

3.2 Distribution of returns

```
p_hist <- ggplot(fx, aes(r)) +
  geom_histogram(aes(y = after_stat(density)), bins = 80,
                 fill = "steelblue4", colour = "white") +
  stat_function(fun = dnorm, args = list(mean = mean(r), sd = sd(r)),
                colour = "firebrick", linewidth = 0.7) +
  labs(x = "Daily return (%)", y = "Density")

p_qq <- ggplot(fx, aes(sample = r)) +
  stat_qq(size = 0.5, colour = "steelblue4") +
  stat_qq_line(colour = "firebrick") +
  labs(x = "Normal quantiles", y = "Sample quantiles")

p_hist + p_qq
```

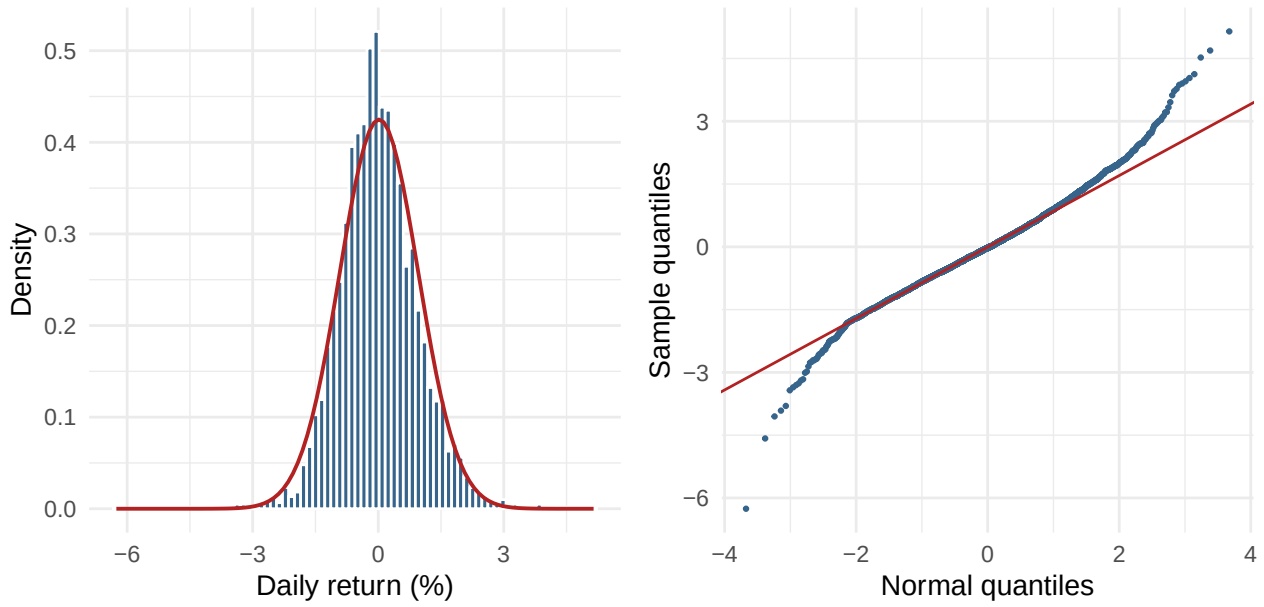


Figure 2: Left: histogram of daily returns with the normal density that has the same mean and variance. Right: normal QQ plot.

The excess kurtosis of 1.88 and the S-shaped QQ plot in Figure 2 show **fat tails**: there are many more very small and very large returns than a normal distribution allows. The Jarque and Bera statistic is 651.4 ($p < 0.001$), so normality is rejected decisively. Part of this is a direct consequence of volatility clustering: a mixture of calm days with a small variance and turbulent days with a large variance has fatter tails than any single normal distribution. Whether returns are still fat-tailed *after* allowing for a changing variance is a separate question that the Student t models answer below.

3.3 Dependence in returns and in squared returns

```
p1 <- fx_ts |> ACF(r, lag_max = 50) |> autoplot() + labs(title = "Returns")
p2 <- fx_ts |> ACF(r^2, lag_max = 50) |> autoplot() + labs(title = "Squared returns")
p3 <- fx_ts |> ACF(abs(r), lag_max = 50) |> autoplot() + labs(title = "Absolute
 ↪  returns")
p1 / p2 / p3
```

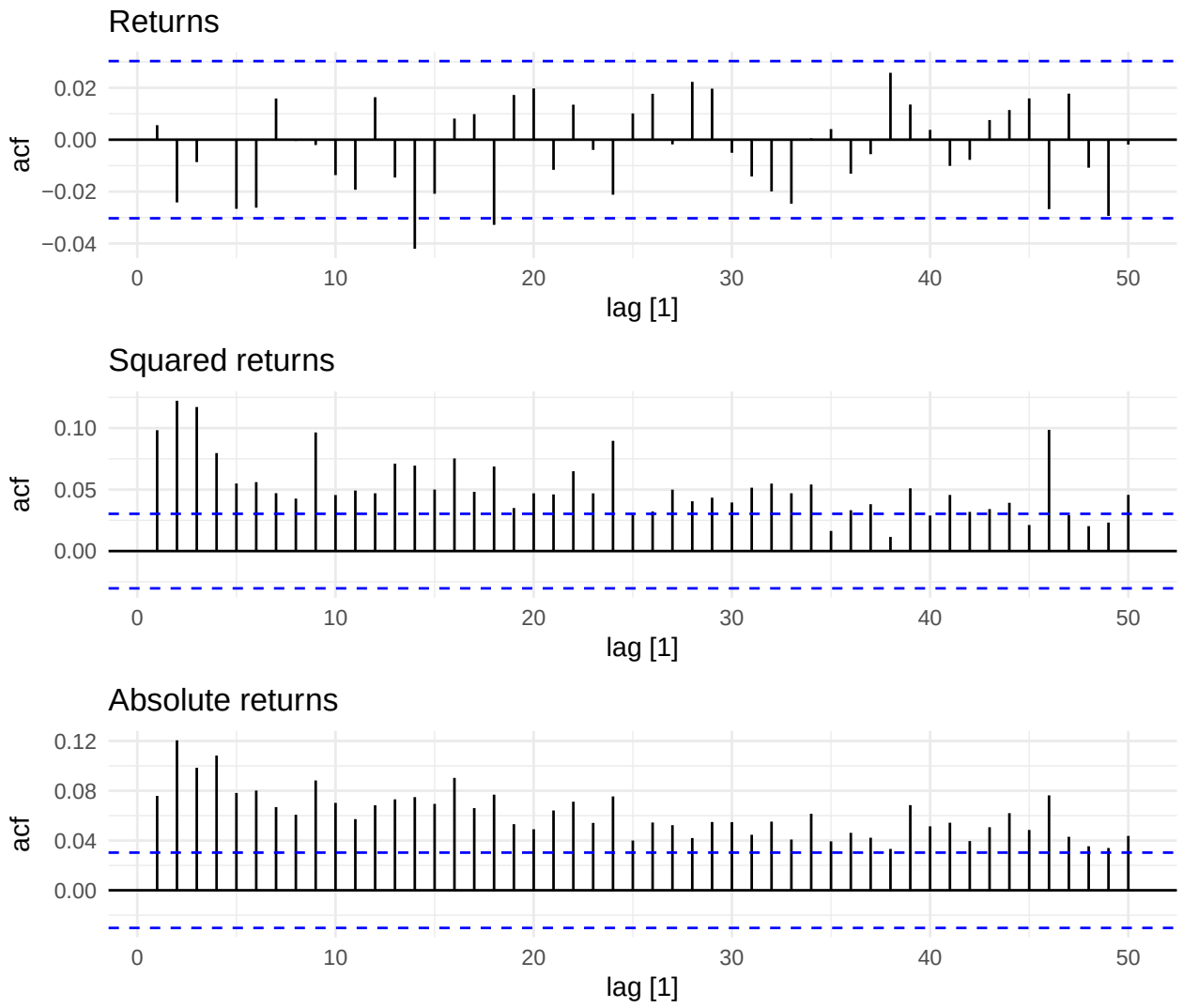


Figure 3: ACF of returns, squared returns and absolute returns, up to 50 trading days.

Figure 3 is the key plot of the project. The returns themselves have almost no autocorrelation: the largest absolute autocorrelation in the first 20 lags is only 0.042. The squared and absolute returns, in contrast, have positive autocorrelations that are significant at every lag shown and decay very slowly. This is the signature of volatility clustering: the *direction* of tomorrow's move is close to unpredictable, but its *size* depends strongly on recent sizes. The Ljung and Box test on 20 lags gives Q = 31.6 (p 0.048) for the returns and Q = 417.2 ($p < 0.001$) for the squared returns.

4 Testing for ARCH effects

The ARCH LM test of Engle regresses the squared demeaned returns on q of their own lags,

$$e_t^2 = a_0 + a_1 e_{t-1}^2 + \cdots + a_q e_{t-q}^2 + u_t,$$

and uses $LM = TR^2 \sim \chi^2_q$ under the null hypothesis of no ARCH effects. I implemented the test directly rather than using a package, so that each step is visible.

```
arch_lm <- function(e, q = 12) {
  e2 <- e^2
  n  <- length(e2)
  X  <- sapply(1:q, function(k) e2[(q - k + 1):(n - k)])   # lagged squares
  y  <- e2[(q + 1):n]
  r2 <- summary(lm(y ~ X))$r.squared
  stat <- length(y) * r2
  c(statistic = stat, df = q, p.value = pchisq(stat, q, lower.tail = FALSE))
}

arch_raw <- arch_lm(r - mean(r), q = 12)
arch_raw
```

```
   statistic           df      p.value
1.781493e+02 1.200000e+01 1.022693e-31
```

With 12 lags the statistic is 178.1 on 12 degrees of freedom ($p < 0.001$). The test rejects the null hypothesis of a constant variance, so a model for the conditional variance is needed. Fitting a constant variance model here would produce prediction intervals that are too wide in calm periods and far too narrow in turbulent periods, which is exactly when accurate intervals matter most.

5 Models

5.1 The Box and Jenkins loop for returns

The project follows the Box and Jenkins workflow from the notes, applied to both equations of the model:

1. **Plot the data:** done in the exploratory analysis.
2. **Difference until approximately stationary:** working with log returns is the first difference of the log exchange rate. The returns fluctuate around a constant mean, so no further differencing is needed.
3. **Read the ACF and PACF to identify candidate orders:** for the *mean* equation the ACF of the returns is close to zero at all lags, so a constant mean, ARMA(0,0), is used. For the *variance* equation the slowly decaying ACF of squared returns suggests that a low order ARCH model will not be enough and that GARCH(1,1) is the natural candidate, since it is an ARCH(∞) model with geometrically decaying weights.
4. **Estimate by maximum likelihood:** with `rugarch`, and with my own code for GARCH(1,1) (Section 6).
5. **Diagnose residuals against the white noise standard:** the standardised residuals $\hat{z}_t = \hat{\varepsilon}_t / \hat{\sigma}_t$ should show no autocorrelation (mean equation adequate) and their squares should show no autocorrelation (variance equation adequate).
6. **Forecast only if diagnostics pass; otherwise return to step 3.**

Finally, a model that survives the loop must still beat the benchmarks out of sample (Section 7).

5.2 Candidate models

All models have a constant mean, $r_t = \mu + \varepsilon_t$ with $\varepsilon_t = \sigma_t z_t$, and differ in the variance equation and in the distribution of z_t:

Table 3: The four candidate models.

Model	Variance equation	Distribution of z_t
ARCH(1)	$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2$	Normal
GARCH(1,1) normal	$\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$	Normal
GARCH(1,1) t	as above	Standardised Student t with ν degrees of freedom
GJR-GARCH(1,1) t	$\sigma_t^2 = \omega + (\alpha + \gamma I_{t-1})\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$, with $I_{t-1} = 1$ if $\varepsilon_{t-1} < 0$	Standardised Student t

The GJR model allows good and bad news to affect volatility differently. For a share price the usual finding is that falls (negative returns) raise volatility more than rises. For USD/ZAR the interpretation is reversed: a *negative* return is a *strengthening* of the rand, so $\gamma > 0$ would mean that rand appreciation raises volatility more, and $\gamma < 0$ that rand depreciation raises volatility more.

```
make_spec <- function(model = "sGARCH", order = c(1, 1), dist = "norm") {
  ugarchspec(
    variance.model     = list(model = model, garchOrder = order),
    mean.model         = list(armaOrder = c(0, 0), include.mean = TRUE),
    distribution.model = dist
  )
}

specs <- list(
  "ARCH(1) normal"    = make_spec(order = c(1, 0)),
  "GARCH(1,1) normal" = make_spec(),
  "GARCH(1,1) t"      = make_spec(dist = "std"),
  "GJR-GARCH(1,1) t"  = make_spec(model = "gjrGARCH", dist = "std")
)
dists <- c("norm", "norm", "std", "std")
names(dists) <- names(specs)

fits <- lapply(specs, function(sp) ugarchfit(spec = sp, data = r, solver = "hybrid"))
```

5.3 Estimates

```
coef_long <- bind_rows(lapply(names(fits), function(m) {
  mc <- fits[[m]]@fit$matcoef
  tibble(model = m, term = rownames(mc),
         value = paste0(formatC(mc[, 1], format = "f", digits = 4), " (",
                        formatC(mc[, 2], format = "f", digits = 4), ")"))
}))
```

```
coef_long |>
  pivot_wider(names_from = model, values_from = value, values_fill = "") |>
  kable()
```

Table 4: Parameter estimates with standard errors in brackets. shape is the Student t degrees of freedom; gamma1 is the GJR asymmetry term.

term	ARCH(1) normal	GARCH(1,1) normal	GARCH(1,1) t	GJR-GARCH(1,1) t
mu	0.0156 (0.0142)	0.0148 (0.0132)	-0.0006 (0.0131)	0.0099 (0.0132)
omega	0.7875 (0.0215)	0.0104 (0.0034)	0.0112 (0.0041)	0.0087 (0.0019)
alpha1	0.1067 (0.0188)	0.0469 (0.0070)	0.0451 (0.0081)	0.0571 (0.0053)
beta1		0.9418 (0.0093)	0.9423 (0.0111)	0.9562 (0.0021)
shape			10.9808 (1.6150)	11.3155 (1.6806)
gamma1				-0.0507 (0.0087)

```
ic_tbl <- bind_rows(lapply(names(fits), function(m) {
  f  <- fits[[m]]
  ic <- infocriteria(f)
  pers <- persistence(f)
  tibble(Model = m,
         `Log likelihood` = likelihood(f),
         AIC = ic[1], BIC = ic[2],
         Persistence = pers,
         `Half-life (days)` = log(0.5) / log(pers))
}))

kable(ic_tbl, digits = c(0, 1, 4, 4, 4, 1))
best_bic <- ic_tbl$Model[which.min(ic_tbl$BIC)]
```

Table 5: Log likelihood, information criteria (per observation, smaller is better), persistence and the half-life of a volatility shock in trading days.

Model	Log likelihood	AIC	BIC	Persistence	Half-life (days)
ARCH(1) normal	-5649.7	2.6995	2.7040	0.1067	0.3
GARCH(1,1) normal	-5516.4	2.6363	2.6423	0.9887	61.0
GARCH(1,1) t	-5482.3	2.6205	2.6280	0.9874	54.7
GJR-GARCH(1,1) t	-5468.0	2.6141	2.6232	0.9880	57.6

Interpreting the GARCH(1,1) t fit. The ARCH coefficient $\hat{\alpha} = 0.045$ measures how strongly volatility reacts to yesterday's shock, and $\hat{\beta} = 0.942$ measures how much of yesterday's volatility carries over. Their sum, the persistence, is 0.9874, very close to 1. A volatility shock therefore dies out slowly, with a half-life of about 55 trading days. The implied long-run (unconditional) standard deviation is 0.945% a day, or 15.0% a year, and forecasts of volatility revert towards this level.

Normal or Student t errors? Even after allowing for a changing variance, the estimated degrees of freedom are $\hat{\nu} = 11.0$, far from the normal limit ($\nu \to \infty$). The log likelihood improves by 34.1 when moving from normal to t errors for the same variance equation, for just one extra parameter, so the

standardised returns are still fat-tailed. This matters for risk: a normal model will understate the probability of extreme days.

Asymmetry. The GJR term is $\hat{\gamma}$ = -0.051 ($p < 0.001$). It is negative and significant, so shocks that weaken the rand (positive returns) raise volatility more than shocks that strengthen it. This is the reverse of the leverage effect in share prices, and is typical of emerging market currencies, where sudden depreciations are associated with stress and capital outflows. By BIC the preferred model is the **GJR-GARCH(1,1) t** model.

5.4 Residual diagnostics (Box and Jenkins step 5)

```
n_garch_par <- c("ARCH(1) normal" = 1, "GARCH(1,1) normal" = 2,
                 "GARCH(1,1) t" = 2, "GJR-GARCH(1,1) t" = 3)

diag_tbl <- bind_rows(lapply(names(fits), function(m) {
  z <- as.numeric(residuals(fits[[m]], standardize = TRUE))
  lbz  <- Box.test(z,   lag = 20, type = "Ljung-Box")
  lbz2 <- Box.test(z^2, lag = 20, type = "Ljung-Box", fitdf = n_garch_par[[m]])
  alm  <- arch_lm(z, q = 12)
  tibble(Model = m,
         `LB z (p)` = lbz$p.value,
         `LB z^2 (p)` = lbz2$p.value,
         `ARCH LM on z (p)` = unname(alm["p.value"]),
         `Excess kurtosis of z` = mean((z - mean(z))^4) / var(z)^2 - 3)
}))

diag_tbl <- diag_tbl |>
  mutate(Passes = if_else(`LB z^2 (p)` > 0.05 & `ARCH LM on z (p)` > 0.05,
                          "Yes", "No"))

kable(diag_tbl, digits = 3)
```

Table 6: Diagnostics on the standardised residuals: Ljung and Box p-values for z and z squared (20 lags; the test on z squared has its degrees of freedom reduced by the number of variance parameters), the ARCH LM test on z (12 lags), and the excess kurtosis of z. A model passes if no ARCH structure remains in z.

Model	LB z (p)	LB z^2 (p)	ARCH LM on z (p)	Excess kurtosis of z	Passes
ARCH(1) normal	0.112	0.000	0.000	1.566	No
GARCH(1,1) normal	0.289	0.721	0.411	0.904	Yes
GARCH(1,1) t	0.280	0.696	0.366	0.911	Yes
GJR-GARCH(1,1) t	0.268	0.665	0.439	1.011	Yes

The two Ljung and Box tests have different jobs, as the notes stress: autocorrelation left in $\hat{z}_t$ would send me back to the *mean* equation, while autocorrelation left in $\hat{z}_t^2$ sends me back to the *variance* equation.

For ARCH(1) the test on $\hat{z}_t^2$ gives p < 0.001 and the ARCH LM test on $\hat{z}_t$ gives p < 0.001. A single lag of the squared shock cannot reproduce the slow decay seen in the ACF of squared returns, so ARCH(1) fails step 5. Following the loop, I returned to step 3 and moved to GARCH(1,1), which adds the lagged variance and so gives the model a long memory with only one extra parameter. ARCH(1) is kept in the comparison only as a reference point; it is not used for forecasting.

All three GARCH(1,1) models pass: no significant autocorrelation remains in the squared standardised residuals, so the variance equation is adequate and the models can be used to forecast (step 6). The test on $\hat{z}_t$ itself gives p-values of 0.112, 0.289, 0.280, 0.268 across the four models, so the constant mean is adequate.

```
fx_sig <- fx |> mutate(sigma = as.numeric(sigma(fits[["GARCH(1,1) t"]])))

ggplot(fx_sig, aes(date)) +
  geom_line(aes(y = r), colour = "grey65", linewidth = 0.2) +
  geom_line(aes(y = 2 * sigma), colour = "firebrick", linewidth = 0.4) +
  geom_line(aes(y = -2 * sigma), colour = "firebrick", linewidth = 0.4) +
  labs(x = NULL, y = "Return (%)")
```

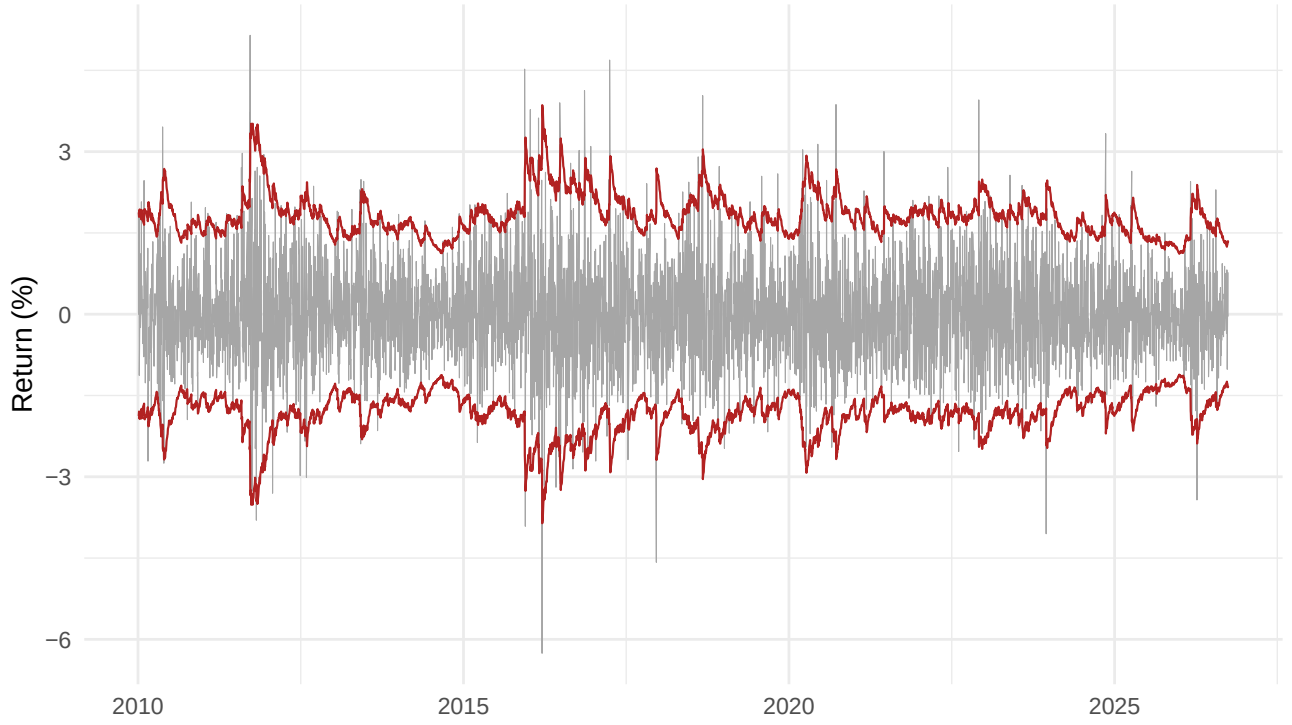


Figure 4: Daily returns with plus and minus two conditional standard deviations from the GARCH(1,1) t model.

```
z_n <- as.numeric(residuals(fits[["GARCH(1,1) normal"]], standardize = TRUE))
z_t <- as.numeric(residuals(fits[["GARCH(1,1) t"]], standardize = TRUE))
pp  <- ppoints(n)

# quantiles of a Student t scaled to unit variance
qstd_unit <- function(p, nu) qt(p, df = nu) * sqrt((nu - 2) / nu)
```

```r
qq_df <- bind_rows(
  tibble(model = "GARCH(1,1) normal", theo = qnorm(pp), samp = sort(z_n)),
  tibble(model = "GARCH(1,1) t", theo = qstd_unit(pp, nu_g), samp = sort(z_t))
)

ggplot(qq_df, aes(theo, samp)) +
  geom_abline(colour = "firebrick") +
  geom_point(size = 0.5, colour = "steelblue4") +
  facet_wrap(vars(model), scales = "free") +
  labs(x = "Theoretical quantiles", y = "Standardised residual quantiles")
```

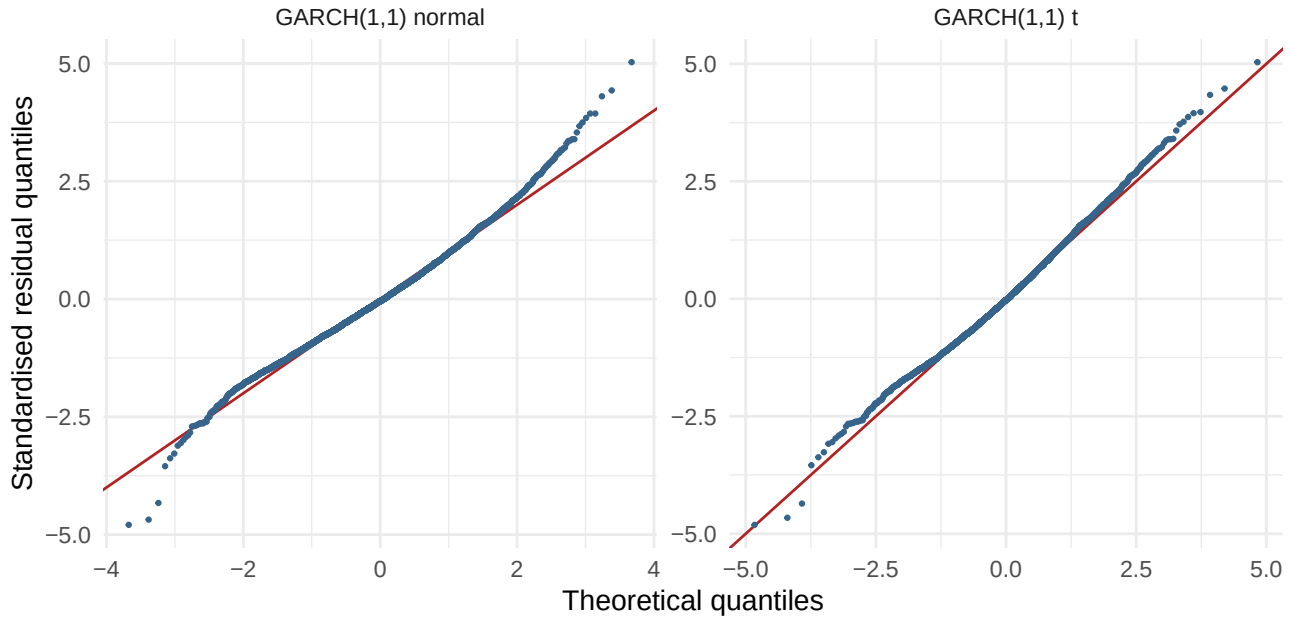


Figure 5: QQ plots of the standardised residuals of the GARCH(1,1) normal model against the normal distribution (left) and of the GARCH(1,1) t model against its fitted Student t distribution (right).

The conditional standard deviation band in Figure 4 widens in the turbulent periods and narrows in calm periods, which a constant variance model cannot do. The QQ plots in Figure 5 show that the normal model's residuals still have heavy tails, while the residuals of the t model lie much closer to the line, confirming the choice of the Student t distribution.

6 My own implementation of GARCH(1,1)

To make sure I understand what `rugarch` is doing, I wrote the GARCH(1,1) likelihood myself. With Gaussian errors the conditional density of r_t given the past is $N(\mu, \sigma_t^2)$, so by the prediction error decomposition the log likelihood is

$$\ell(\mu, \omega, \alpha, \beta) = -\frac{1}{2}\sum_{t=1}^{T}\left[\log(2\pi) + \log\sigma_t^2 + \frac{(r_t-\mu)^2}{\sigma_t^2}\right], \qquad \sigma_t^2 = \omega + \alpha(r_{t-1}-\mu)^2 + \beta\sigma_{t-1}^2.$$

The recursion is started at $\sigma_1^2 = \frac{1}{T}\sum(r_t-\mu)^2$. The constraints $\omega > 0$, $\alpha, \beta \geq 0$ and $\alpha + \beta < 1$ (needed for a finite unconditional variance) are imposed by optimising over unconstrained *working* parameters,

$$\omega = e^{\theta_2}, \quad \alpha = \frac{e^{\theta_3}}{1 + e^{\theta_3} + e^{\theta_4}}, \quad \beta = \frac{e^{\theta_4}}{1 + e^{\theta_3} + e^{\theta_4}},$$

Standard errors come from the inverse of the numerical Hessian of the negative log likelihood. The Hessian is computed on the working scale, because a numerical derivative on the natural scale can step outside the region $\alpha + \beta < 1$ when the estimates are close to it (which they are for persistent series like this one), giving a negative variance and an undefined likelihood. The working scale covariance V_θ is then mapped to the natural scale with the delta method, $V = JV_\theta J^\top$, where J is the Jacobian of the transformation.

```
garch11_filter <- function(par, r) {
  mu <- par[1]; omega <- par[2]; alpha <- par[3]; beta <- par[4]
  e  <- r - mu
  s2 <- numeric(length(r))
  s2[1] <- mean(e^2)
  for (t in 2:length(r)) s2[t] <- omega + alpha * e[t - 1]^2 + beta * s2[t - 1]
  list(e = e, sigma2 = s2)
}

garch11_nll <- function(par, r) {
  f <- garch11_filter(par, r)
  # guard against invalid variances (can only happen at the very edge of
  # the parameter space during numerical differentiation)
  if (any(!is.finite(f$sigma2)) || any(f$sigma2 <= 0)) return(1e10)
  0.5 * sum(log(2 * pi) + log(f$sigma2) + f$e^2 / f$sigma2)
}

to_natural <- function(theta) {
  d <- 1 + exp(theta[3]) + exp(theta[4])
  c(mu = theta[1], omega = exp(theta[2]),
    alpha = exp(theta[3]) / d, beta = exp(theta[4]) / d)
}

to_working <- function(par) {
  rest <- 1 - par[3] - par[4]
  c(par[1], log(par[2]), log(par[3] / rest), log(par[4] / rest))
}

# numerical Jacobian by central differences, used for the delta method
jacobian_num <- function(f, x, h = 1e-6) {
  J <- matrix(0, length(f(x)), length(x))
  for (j in seq_along(x)) {
    xp <- x; xm <- x
```

```
    xp[j] <- x[j] + h; xm[j] <- x[j] - h
    J[, j] <- (f(xp) - f(xm)) / (2 * h)
  }
  J
}

fit_garch11 <- function(r, start = c(mean(r), 0.1 * var(r), 0.05, 0.90), se = TRUE) {
  nll_w <- function(th) garch11_nll(to_natural(th), r)
  opt <- optim(to_working(start), nll_w, method = "BFGS",
               control = list(maxit = 2000, reltol = 1e-10))
  est <- to_natural(opt$par)

  se_nat <- rep(NA_real_, 4)
  if (se) {
    # Hessian on the unconstrained working scale, then the delta method:
    # Var(natural) = J Var(working) J', with J the Jacobian of to_natural()
    H_w <- optimHess(opt$par, nll_w)
    V_w <- tryCatch(solve(H_w), error = function(e) matrix(NA_real_, 4, 4))
    J   <- jacobian_num(to_natural, opt$par)
    se_nat <- sqrt(pmax(diag(J %*% V_w %*% t(J)), 0))
  }

  list(estimate = est,
       se = setNames(se_nat, names(est)),
       loglik = -opt$value,
       convergence = opt$convergence,
       sigma2 = garch11_filter(est, r)$sigma2)
}

own <- fit_garch11(r)

rg <- fits[["GARCH(1,1) normal"]]@fit$matcoef

tibble(
  Parameter = c("mu", "omega", "alpha", "beta", "log likelihood"),
  `Own code` = c(own$estimate, own$loglik),
  `Own s.e.` = c(own$se, NA),
  rugarch = c(rg[, 1], likelihood(fits[["GARCH(1,1) normal"]])),
  `rugarch s.e.` = c(rg[, 2], NA)
) |>
  kable(digits = 4)
sigma_cor <- cor(sqrt(own$sigma2), as.numeric(sigma(fits[["GARCH(1,1) normal"]])))
```

Table 7: My own GARCH(1,1) estimates compared with rugarch (normal errors), with standard errors.

Parameter	Own code	Own s.e.	rugarch	rugarch s.e.
mu	0.0148	0.0132	0.0148	0.0132
omega	0.0104	0.0032	0.0104	0.0034
alpha	0.0469	0.0064	0.0469	0.0070

Table 7: My own GARCH(1,1) estimates compared with rugarch (normal errors), with standard errors.

Parameter	Own code	Own s.e.	rugarch	rugarch s.e.
beta	0.9418	0.0083	0.9418	0.0093
log likelihood	-5516.3598	NA	-5516.3598	NA

My estimates agree with `rugarch` to two or three decimal places, the log likelihoods differ by only 0.00, and the two fitted volatility paths have a correlation of 1.0000. The small differences come from how the recursion is started (the value used for σ_1^2) and from the optimiser's stopping rules, not from a difference in the model.

6.1 A small simulation check

Agreement with a package shows that two implementations agree, not that either is right. As a second check, I simulated 100 series of the same length as the data from a GARCH(1,1) with known parameters (set equal to the `rugarch` estimates) and refitted each one with my code.

```
sim_garch11 <- function(n, mu, omega, alpha, beta, burn = 500) {
  N <- n + burn
  z <- rnorm(N); e <- s2 <- numeric(N)
  s2[1] <- omega / (1 - alpha - beta)
  e[1]  <- sqrt(s2[1]) * z[1]
  for (t in 2:N) {
    s2[t] <- omega + alpha * e[t - 1]^2 + beta * s2[t - 1]
    e[t]  <- sqrt(s2[t]) * z[t]
  }
  mu + e[(burn + 1):N]
}

true_par <- rg[, 1]
names(true_par) <- c("mu", "omega", "alpha", "beta")

set.seed(42)
n_sim <- 100
sim_est <- t(replicate(n_sim, {
  y <- sim_garch11(n, true_par["mu"], true_par["omega"], true_par["alpha"],
 ↪  true_par["beta"])
  fit_garch11(y, se = FALSE)$estimate   # standard errors not needed here
}))

tibble(
  Parameter = names(true_par),
  True = true_par,
  `Mean estimate` = colMeans(sim_est),
  Bias = colMeans(sim_est) - true_par,
  `Empirical s.d.` = apply(sim_est, 2, sd),
  `Hessian s.e. (data fit)` = own$se
) |>
  kable(digits = 4)
```

Table 8: Simulation check (100 replications, same length as the data): true values, mean estimate, bias, and the empirical standard deviation of the estimates compared with the average Hessian based standard error from the real-data fit.

Parameter	True	Mean estimate	Bias	Empirical s.d.	Hessian s.e. (data fit)
mu	0.0148	0.0144	-0.0004	0.0133	0.0132
omega	0.0104	0.0054	-0.0050	0.0055	0.0032
alpha	0.0469	0.0252	-0.0216	0.0243	0.0064
beta	0.9418	0.9692	0.0274	0.0297	0.0083

The estimates are centred on the true values with small bias, and the spread of the estimates across replications is of the same size as the Hessian based standard errors, so both the estimator and its standard errors behave as the theory says they should.

7 Out-of-sample evaluation: Value-at-Risk backtest

7.1 Design

In-sample fit does not answer the research question, which is about forecasts. The last 1000 trading days (from 30 September 2022 to 30 September 2026) are used as a test period. For each day in the test period, every model produces a one-day-ahead forecast of the mean and standard deviation using only data up to the previous day. The GARCH models are re-estimated every 25 trading days on an expanding window, which keeps the evaluation free of temporal leakage.

From each forecast distribution I compute the VaR at four levels: the 1% and 5% quantiles (losses for someone holding dollars when the rand strengthens) and the 95% and 99% quantiles (losses for someone who needs dollars, such as an importer, when the rand weakens). For Student t models,

$$\mathrm{VaR}_{p,t} = \hat{\mu}_{t|t-1} + \hat{\sigma}_{t|t-1}\, t_{\nu}^{-1}(p)\sqrt{(\nu-2)/\nu},$$

where the last factor rescales the t distribution to unit variance.

Two benchmarks without a GARCH variance are included, in the spirit of the benchmark methods in the notes:

- **Historical simulation (HS):** the VaR is the empirical quantile of the previous 250 returns.
- **Rolling normal:** a normal distribution with the mean and variance of the previous 250 returns.

The ARCH(1) model failed the residual diagnostics, so following step 6 it is not part of the forecast comparison.

```
roll_models <- c("GARCH(1,1) normal", "GARCH(1,1) t", "GJR-GARCH(1,1) t")

rolls <- lapply(specs[roll_models], function(sp) {
  rl <- ugarchroll(sp, data = r, n.ahead = 1, forecast.length = n_test,
                   refit.every = refit_every, refit.window = "recursive",
                   solver = "hybrid", calculate.VaR = FALSE)
  if (convergence(rl) != 0) rl <- resume(rl, solver = "gosolnp")
```

```
  rl
})

levels_p  <- c(0.01, 0.05, 0.95, 0.99)
test_idx  <- (n - n_test + 1):n
realised  <- r[test_idx]

var_quantile <- function(mu, sigma, p, dist, shape = NULL) {
  if (dist == "norm") mu + sigma * qnorm(p) else mu + sigma * qstd_unit(p, shape)
}

# GARCH model forecasts
fc_list <- lapply(roll_models, function(m) {
  d <- as.data.frame(rolls[[m]], which = "density")
  out <- tibble(model = m, t = seq_len(n_test), mu = d$Mu, sigma = d$Sigma,
                realised = realised)
  for (p in levels_p) {
    out[[paste0("q", p)]] <- var_quantile(d$Mu, d$Sigma, p, dists[[m]], d$Shape)
  }
  out
})

# Benchmarks
bench_hs <- tibble(model = "Historical simulation", t = seq_len(n_test),
                   realised = realised)
bench_rn <- tibble(model = "Rolling normal", t = seq_len(n_test),
                   realised = realised)
past <- lapply(test_idx, function(i) r[(i - hs_window):(i - 1)])
bench_rn$mu    <- sapply(past, mean)
bench_rn$sigma <- sapply(past, sd)
bench_hs$mu    <- bench_rn$mu
bench_hs$sigma <- bench_rn$sigma
for (p in levels_p) {
  bench_hs[[paste0("q", p)]] <- sapply(past, quantile, probs = p, names = FALSE)
  bench_rn[[paste0("q", p)]] <- bench_rn$mu + bench_rn$sigma * qnorm(p)
}

fc_all <- bind_rows(fc_list, list(bench_hs, bench_rn)) |>
  mutate(date = fx$date[test_idx][t])
```

7.2 Backtest tests

For each model and level, a *hit* is a day on which the realised return falls beyond the VaR (below it for the 1% and 5% levels, above it for the 95% and 99% levels). Under a correct model the hits are independent Bernoulli(p^*) variables, where $p^* = p$ for the lower tail and $p^* = 1 - p$ for the upper tail. I implemented two standard likelihood ratio tests.

- **Kupiec unconditional coverage test:** is the hit rate equal to p^*? With x hits in T days and $\hat{\pi} = x/T$, $LR_{uc} = -2\left[(T-x)\log(1-p^*) + x\log p^* - (T-x)\log(1-\hat{\pi}) - x\log\hat{\pi}\right] \sim \chi^2_1$.

- **Christoffersen conditional coverage test:** adds a test of independence, comparing a first-order Markov chain for the hits with an independent sequence, so that clustered hits are detected. $LR_{cc} = LR_{uc} + LR_{ind} \sim \chi^2_2$.

```
kupiec_test <- function(hits, p) {
  n <- length(hits); x <- sum(hits); phat <- x / n
  ll0 <- (n - x) * log(1 - p) + x * log(p)
  ll1 <- if (x == 0 || x == n) 0 else (n - x) * log(1 - phat) + x * log(phat)
  lr  <- -2 * (ll0 - ll1)
  c(hits = x, expected = n * p, LR_uc = lr, p_uc = pchisq(lr, 1, lower.tail = FALSE))
}

christoffersen_test <- function(hits, p) {
  h0 <- hits[-length(hits)]; h1 <- hits[-1]
  n00 <- sum(h0 == 0 & h1 == 0); n01 <- sum(h0 == 0 & h1 == 1)
  n10 <- sum(h0 == 1 & h1 == 0); n11 <- sum(h0 == 1 & h1 == 1)
  pi01 <- n01 / (n00 + n01)
  pi11 <- if ((n10 + n11) > 0) n11 / (n10 + n11) else 0
  pi1  <- (n01 + n11) / (n00 + n01 + n10 + n11)
  xlogy <- function(x, y) ifelse(x == 0, 0, x * log(y))
  ll_ind <- xlogy(n00, 1 - pi01) + xlogy(n01, pi01) +
            xlogy(n10, 1 - pi11) + xlogy(n11, pi11)
  ll_dep <- xlogy(n00 + n10, 1 - pi1) + xlogy(n01 + n11, pi1)
  lr_ind <- -2 * (ll_dep - ll_ind)
  lr_cc  <- kupiec_test(hits, p)[["LR_uc"]] + lr_ind
  c(p_ind = pchisq(lr_ind, 1, lower.tail = FALSE),
    p_cc  = pchisq(lr_cc, 2, lower.tail = FALSE))
}

# quantile (tick) loss: the proper scoring rule for a p quantile
tick_loss <- function(y, q, p) mean((p - (y < q)) * (y - q))

bt <- expand_grid(model = unique(fc_all$model), p = levels_p) |>
  rowwise() |>
  mutate(res = list({
    d <- fc_all[fc_all$model == model, ]
    q <- d[[paste0("q", p)]]
    y <- d$realised
    p_star <- if (p < 0.5) p else 1 - p
    hits <- if (p < 0.5) as.integer(y < q) else as.integer(y > q)
    k <- kupiec_test(hits, p_star)
    ch <- christoffersen_test(hits, p_star)
    tibble(Hits = k[["hits"]], Expected = k[["expected"]],
           `Kupiec p` = k[["p_uc"]], `Christoffersen p` = ch[["p_cc"]],
           `Tick loss` = tick_loss(y, q, p))
  })) |>
  ungroup() |>
  unnest(res) |>
  mutate(Level = paste0(100 * p, "%"))

bt |>
```

```
  select(Level, Model = model, Hits, Expected, `Kupiec p`, `Christoffersen p`, `Tick
   ↪  loss`) |>
  arrange(Level, `Tick loss`) |>
  kable(digits = c(0, 0, 0, 0, 3, 3, 5))
```

Table 9: VaR backtest over the test period. Expected is the number of hits a correct model would produce. p-values below 0.05 indicate a rejected model. Tick loss is the average quantile loss (smaller is better) and allows the models to be ranked at each level.

Level	Model	Hits	Expected	Kupiec p	Christoffersen p	Tick loss
1%	GARCH(1,1) normal	7	10	0.314	0.573	0.02334
1%	GJR-GARCH(1,1) t	4	10	0.030	0.094	0.02346
1%	GARCH(1,1) t	4	10	0.030	0.094	0.02389
1%	Rolling normal	7	10	0.314	0.573	0.02409
1%	Historical simulation	11	10	0.754	0.842	0.02421
5%	GJR-GARCH(1,1) t	29	50	0.001	0.004	0.07911
5%	GARCH(1,1) t	29	50	0.001	0.004	0.08063
5%	GARCH(1,1) normal	29	50	0.001	0.004	0.08063
5%	Rolling normal	34	50	0.014	0.037	0.08278
5%	Historical simulation	55	50	0.475	0.660	0.08305
95%	GARCH(1,1) t	49	50	0.884	0.034	0.09059
95%	GARCH(1,1) normal	42	50	0.233	0.052	0.09075
95%	GJR-GARCH(1,1) t	54	50	0.566	0.202	0.09124
95%	Rolling normal	46	50	0.557	0.055	0.09302
95%	Historical simulation	46	50	0.557	0.168	0.09378
99%	GARCH(1,1) normal	10	10	1.000	0.904	0.02657
99%	Historical simulation	12	10	0.538	0.715	0.02658
99%	GARCH(1,1) t	9	10	0.746	0.875	0.02660
99%	GJR-GARCH(1,1) t	11	10	0.754	0.842	0.02671
99%	Rolling normal	18	10	0.022	0.009	0.02731

7.3 Variance forecast accuracy

The VaR tests look at the tails. To compare the variance forecasts as a whole I use the QLIKE loss, $L_t = \log \hat{\sigma}^2_{t|t-1} + r_t^2 / \hat{\sigma}^2_{t|t-1}$, with the squared return as a noisy proxy for the true variance. QLIKE is robust to that noise in the sense that it ranks forecasts correctly in expectation, and it penalises under-prediction of variance more heavily than over-prediction, which is the right asymmetry for risk management.

```
qlike_tbl <- fc_all |>
  filter(model != "Historical simulation") |>
  group_by(Model = model) |>
  summarise(QLIKE = mean(log(sigma^2) + (realised - mu)^2 / sigma^2), .groups =
   ↪  "drop") |>
  arrange(QLIKE)

kable(qlike_tbl, digits = 4)
```

Table 10: Average QLIKE loss of the one-day-ahead variance forecasts over the test period (smaller is better). Historical simulation uses the same rolling variance as the rolling normal benchmark.

Model	QLIKE
GJR-GARCH(1,1) t	0.5623
GARCH(1,1) t	0.5666
GARCH(1,1) normal	0.5674
Rolling normal	0.6070

```
plot_df <- fc_all |> filter(model %in% c("GARCH(1,1) t", "Historical simulation"))
hits_df <- plot_df |>
  filter(model == "GARCH(1,1) t", realised < q0.01 | realised > q0.99)

ggplot(plot_df, aes(date)) +
  geom_line(data = filter(plot_df, model == "GARCH(1,1) t"),
            aes(y = realised), colour = "grey60", linewidth = 0.2) +
  geom_line(aes(y = q0.01, colour = model), linewidth = 0.4) +
  geom_line(aes(y = q0.99, colour = model), linewidth = 0.4) +
  geom_point(data = hits_df, aes(y = realised), colour = "black", size = 1) +
  scale_colour_manual(values = c("GARCH(1,1) t" = "firebrick",
                                 "Historical simulation" = "steelblue")) +
  labs(x = NULL, y = "Return (%)", colour = NULL) +
  theme(legend.position = "bottom")
```

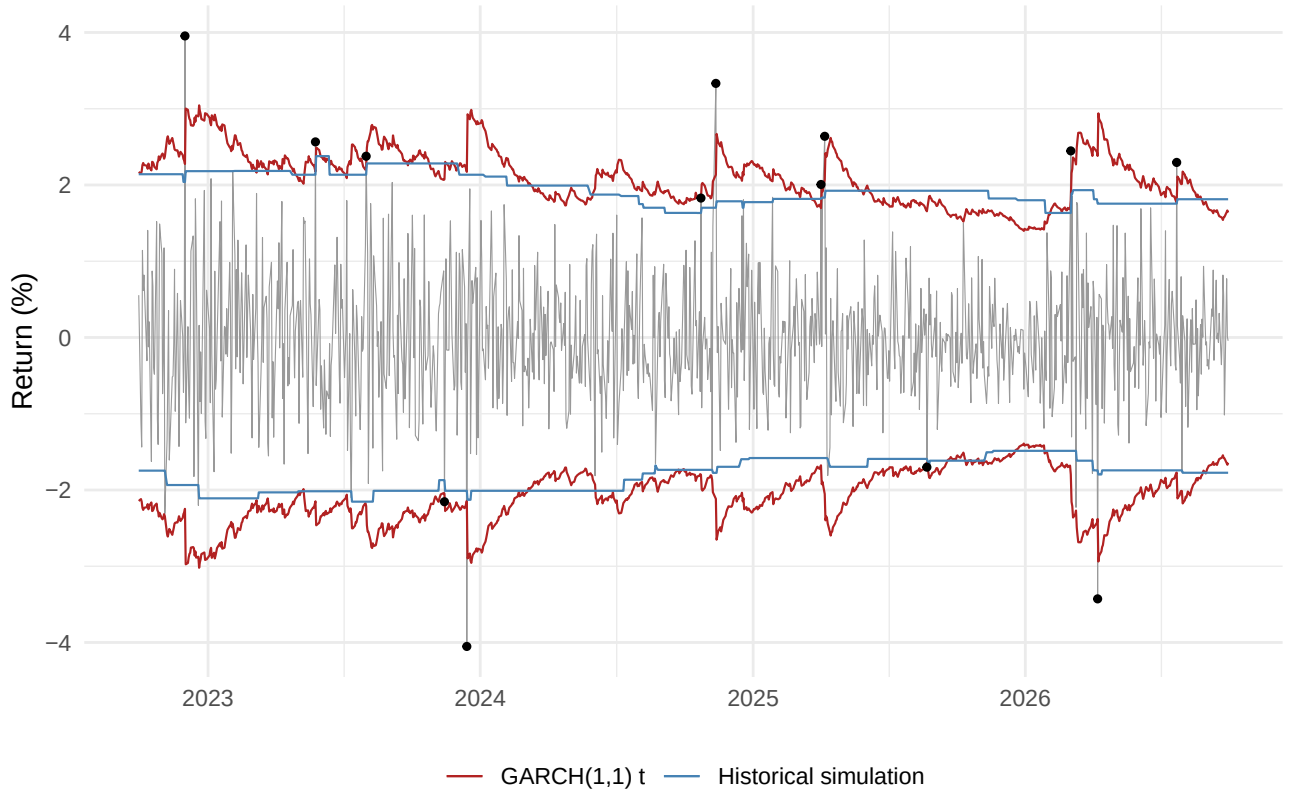


Figure 6: Test period returns with the 1% and 99% one-day VaR from the GARCH(1,1) t model (red) and historical simulation (blue). Points mark returns beyond the GARCH 1% or 99% VaR.

Coverage. Out of the four levels, the number of rejections (Kupiec or Christoffersen test at 5%) is 1 for GARCH(1,1) normal, 3 for GARCH(1,1) t, 2 for GJR-GARCH(1,1) t, 0 for historical simulation and 2 for the rolling normal benchmark. At the 1% level, where a correct model should be exceeded about 10 times, the GARCH(1,1) normal model is exceeded 7 times in the lower tail and 10 times in the upper tail, against 4 and 9 for the GARCH(1,1) t model. The normal model is exceeded more often in the extreme tails, which is the practical consequence of the fat tails found in its residuals.

Clustering of exceedances. The benchmarks react slowly to a change in volatility: a window of 250 days gives the same weight to a return from a year ago as to yesterday's. When volatility jumps, their VaR is too narrow for weeks and the hits bunch together, which is what the Christoffersen test detects; when volatility falls, their VaR stays too wide for a long time (Figure 6). The GARCH VaR bands widen and narrow within days.

Ranking. Averaged over the four levels, the lowest quantile loss is achieved by the **GJR-GARCH(1,1) t** model, and the lowest QLIKE by the **GJR-GARCH(1,1) t** model.

8 Volatility forecasts

```
final_model <- best_bic
fc_final <- ugarchforecast(fits[[final_model]], n.ahead = 20)
sig_path <- as.numeric(sigma(fc_final))
```

```
lr_sd    <- sqrt(uncvariance(fits[[final_model]]))
last_sig <- tail(as.numeric(sigma(fits[[final_model]])), 1)

tibble(h = 1:20, sigma = sig_path) |>
  ggplot(aes(h, sigma)) +
  geom_hline(yintercept = lr_sd, linetype = "dashed", colour = "grey40") +
  geom_line(colour = "firebrick") +
  geom_point(colour = "firebrick", size = 1) +
  labs(x = "Trading days ahead", y = "Conditional s.d. (%)")
```

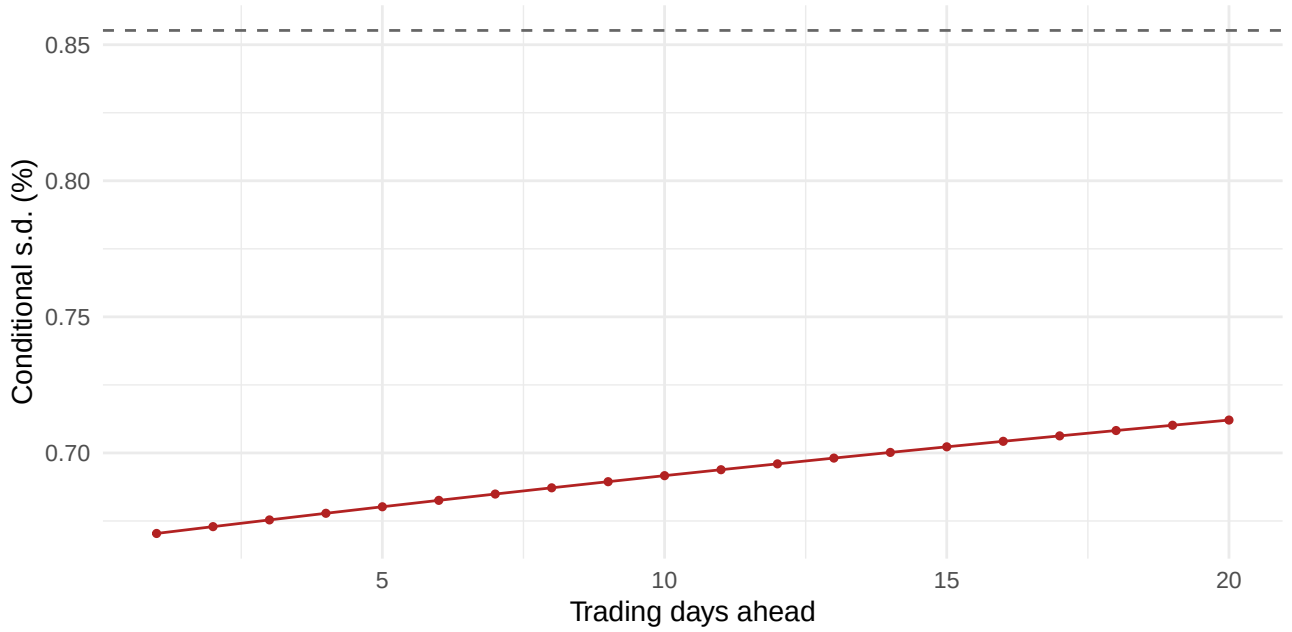


Figure 7: Forecast of the daily conditional standard deviation for the next 20 trading days from the preferred model, with the long-run level (dashed).

Using the preferred GJR-GARCH(1,1) t model fitted to the full sample, the conditional standard deviation on the last day of the sample is 0.679%, and the forecast moves from 0.670% one day ahead to 0.712% twenty days ahead, towards the long-run level of 0.855% (Figure 7). The speed of this reversion is governed by the persistence, as in the notes: beyond one step, volatility forecasts revert geometrically to the unconditional variance, so GARCH forecasts are informative for days to weeks but say little about volatility months ahead. Because the mean return is close to zero, the main value of the model is in these variance forecasts and the intervals built from them, not in point forecasts of the exchange rate.

9 Discussion

9.1 Key findings

1. **The rand shows all the stylised facts of financial returns:** almost no autocorrelation in returns, strong and slowly decaying autocorrelation in squared returns, and fat tails (excess kurtosis 1.9). The ARCH LM test confirms a time-varying variance ($p < 0.001$).

2. **The Box and Jenkins loop rejected ARCH(1)** because ARCH structure remained in its standardised residuals, and led to GARCH(1,1).
3. **Volatility is highly persistent:** $\widehat{\alpha} + \widehat{\beta} = 0.9874$, a half-life of about 55 trading days.
4. **Student t errors are clearly better than normal errors** ($\widehat{\nu} = 11.0$), so even after removing volatility clustering, extreme days are more common than a normal distribution implies.
5. **Asymmetry:** $\widehat{\gamma}$ = -0.051 ($p < 0.001$). There is significant asymmetry in how depreciation and appreciation shocks affect volatility.
6. **Out of sample**, the preferred model by quantile loss is GJR-GARCH(1,1) t and by QLIKE GJR-GARCH(1,1) t. The number of rejected VaR tests (out of four levels) was 3 for GARCH(1,1) t against 0 for historical simulation and 2 for the rolling normal benchmark.

9.2 Issues and limitations

- **Power of the tests.** With 1000 test days, a correct 1% VaR is expected to be exceeded only 10 times, so the Kupiec test has little power to separate a 1% model from, say, a 1.5% model. A model that "passes" is not proven correct.
- **Tests with a large sample.** The residual tests in-sample use more than 4000 observations, so even very small departures from white noise can be significant. I combined the formal tests with the plots and the out-of-sample results rather than relying on p-values alone.
- **Very high persistence.** A persistence close to 1 can be a sign of structural breaks in the level of volatility (for example a calmer period after 2020) rather than true long memory, as discussed in the notes. A model with a regime change in volatility would be a natural alternative, and this is the subject of Project 4.
- **Data choices.** FRED noon rates in New York miss moves that happen during South African trading hours after the New York fix, and dropping US holidays makes some returns cover two days. Neither should affect the conclusions much, but a Johannesburg close series would be a useful check.
- **Refitting every 25 days** rather than daily saves time but means the parameters lag slightly behind; the volatility itself is still updated every day.
- **VaR is only one risk measure.** It says nothing about how large losses are once the VaR is exceeded. Expected shortfall would be the natural extension.

9.3 What I would do differently

I would use a longer test period, add expected shortfall to the backtest, and compare GARCH with a hidden Markov model that lets volatility switch between a small number of regimes (Project 4), using the same returns and the same backtest so that the two approaches can be compared directly.

Reproducibility

All results are produced by the code in this document from `usdzar_fred.csv`. The first render downloads the file from FRED and saves it; later renders reuse the saved file. Random seeds are fixed for the simulation study. The simulation and the rolling forecasts are cached (`cache: true`) because they take a few minutes; delete the `_cache` folder to force them to rerun.

```
R version 4.5.3 (2026-03-11 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_South Africa.utf8  LC_CTYPE=English_South Africa.utf8
[3] LC_MONETARY=English_South Africa.utf8 LC_NUMERIC=C
[5] LC_TIME=English_South Africa.utf8

time zone: Africa/Johannesburg
tzcode source: internal

attached base packages:
[1] parallel  stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
 [1] patchwork_1.3.2   knitr_1.51        rugarch_1.5-6     fable_0.5.0
 [5] feasts_0.5.0      fabletools_0.8.0  ggtime_0.2.0      tsibbledata_0.4.1
 [9] tsibble_1.2.0     ggplot2_4.0.3     lubridate_1.9.5   tidyr_1.3.2
[13] dplyr_1.2.0       tibble_3.3.1      fpp3_1.0.3

loaded via a namespace (and not attached):
 [1] gtable_0.3.6                anytime_0.3.13
 [3] xfun_0.56                   ks_1.15.2
 [5] lattice_0.22-9              numDeriv_2016.8-1.1
 [7] vctrs_0.7.1                 tools_4.5.3
 [9] generics_0.1.4              xts_0.14.2
[11] pkgconfig_2.0.3             Matrix_1.7-4
[13] KernSmooth_2.23-26          RColorBrewer_1.1-3
[15] DistributionUtils_0.6-2     S7_0.2.1
[17] distributional_0.8.1        lifecycle_1.0.5
[19] truncnorm_1.0-9             compiler_4.5.3
[21] farver_2.1.2                spd_2.0-1
[23] codetools_0.2-20            SkewHyperbolic_0.4-2
[25] htmltools_0.5.9             Rsolnp_2.0.1
[27] yaml_2.3.12                 pracma_2.4.6
[29] pillar_1.11.1               nloptr_2.2.1
[31] crayon_1.5.3                MASS_7.3-65
[33] GeneralizedHyperbolic_0.8-7 mclust_6.1.3
[35] parallelly_1.48.0           fracdiff_1.5-4
[37] tidyselect_1.2.1            digest_0.6.39
[39] mvtnorm_1.4-2               future_1.75.0
[41] purrr_1.2.2                 listenv_1.0.0
[43] labeling_0.4.3              fastmap_1.2.0
[45] grid_4.5.3                  cli_3.6.5
```

```
[47] magrittr_2.0.4             future.apply_1.20.2
[49] withr_3.0.3                scales_1.4.0
[51] rappdirs_0.3.4             timechange_0.4.0
[53] rmarkdown_2.31             globals_0.19.1
[55] otel_0.2.0                 zoo_1.8-15
[57] evaluate_1.0.5             rlang_1.1.7
[59] Rcpp_1.1.2                 glue_1.8.0
[61] rstudioapi_0.19.0          jsonlite_2.0.0
[63] R6_2.6.1
```